

Engenharia de Software

Requisitos

Prof. Tiago Piperno Bonetti

Material adaptado de: Marco Tulio Valente
<https://engsoftmoderna.info>

***"A parte mais difícil da
construção de um software é a
definição do que se deve
construir"***

Fred Brooks

Requisitos

O que um sistema **deve fazer** e sob que **restrições**.

"o que um sistema deve fazer": **requisitos funcionais**

- » Suas funcionalidades

"sob que restrições": **requisitos não-funcionais**

- » Desempenho, usabilidade, segurança, disponibilidade, etc

Exemplo:

Sistema de um banco



Quais são os requisitos Funcionais?

Quais são os requisitos Não-funcionais?

Requisitos Funcionais

- » Abrir e fechar conta
- » Fornecer extrato e saldo
- » Realizar transferência
- » Realizar saque
- » etc

Requisitos Não-Funcionais

- » Desempenho: informar o saldo em menos de 5 segundos.
- » Disponibilidade: estar no ar 99.99% do tempo.
- » Tolerância a falhas: continuar operando se o centro de dados cair.
- » Segurança: criptografar dados trocados com as agências.
- » Privacidade: não disponibilizar dados de clientes para terceiros.

Requisitos Não-Funcionais

- » Interoperabilidade: integrar-se com os sistemas do BACEN
- » Capacidade: armazenar dados de 1 milhão de clientes
- » Usabilidade: ter uma versão para deficientes visuais

Engenharia de Requisitos

Engenharia de Requisitos é o nome que se dá ao **conjunto de atividades** relacionadas com a **descoberta, análise, especificação e manutenção dos requisitos** de um sistema.

As **atividades** relacionadas com a descoberta e entendimento dos requisitos de um sistema são chamadas de **Elicitação de Requisitos**.

- » As interações dos desenvolvedores de um sistema com os seus stakeholders, com o objetivo de descobrir e entender os principais requisitos do sistema.

Engenharia de Requisitos

Diversas **técnicas** podem ser usadas para elicitação de requisitos, incluindo:

- » Entrevistas com stakeholders;
- » Aplicação de questionários;
- » Leitura de documentos e formulários da organização que está contratando o sistema;
- » Realização de workshops com os usuários;
- » Implementação de protótipos;
- » Análise de cenários de uso.

Engenharia de Requisitos

Existem ainda técnicas de eliciação de requisitos baseadas em **estudos etnográficos**.

O **desenvolvedor se integre ao ambiente de trabalho** dos stakeholders e **observe** — normalmente, por alguns dias — como ele desenvolve suas atividades.

Essa observação é silenciosa, isto é, o desenvolvedor **não interfere e opina** sobre as tarefas e eventos que estão sendo observados.

Engenharia de Requisitos

Após elicitados, os requisitos devem ser:

- » Documentados;
- » Verificados e validados;
- » Priorizados.

Engenharia de Requisitos

» Documentados;

No caso de desenvolvimento **ágil**, a documentação de requisitos é feita de forma **simplificada**, por meio de **histórias do usuário**.

Em **alguns** projetos ainda se **exige um Documento de Especificação** de Requisitos, no qual todos os requisitos do software que se pretende construir são documentados em linguagem natural.

Engenharia de Requisitos

» Verificados e validados;

O **objetivo** é garantir que eles estejam corretos, precisos, completos, consistentes e verificáveis.

Corretos. Um contra-exemplo é a especificação de forma incorreta da fórmula para remuneração das cadernetas de poupança em um sistema bancário.

Engenharia de Requisitos

Precisos, isto é, não devem ser ambíguos.

Ambiguidade ocorre com mais frequência do que gostaríamos quando usamos a linguagem natural.

*Exemplo: considere essa condição: para ser **aprovado** um aluno precisa obter 60 pontos no semestre ou 60 pontos no Exame Especial e ser frequente.*

Veja que ela admite duas interpretações. A primeira é a seguinte: (60 pontos no semestre ou 60 pontos no Exame Especial) e ser frequente.

*Porém, pode-se interpretar também como: 60 pontos no semestre ou (60 pontos no Exame Especial e ser frequente). Conforme você observou, tivemos que usar parênteses para eliminar a ambiguidade na ordem das operações **e e ou**.*

Engenharia de Requisitos

Completos. Isto é, não podemos esquecer de especificar certos requisitos, principalmente se eles forem importantes no sistema que se pretende construir.

Consistentes. Um contra-exemplo ocorre quando um stakeholder afirma que a disponibilidade do sistema deve ser 99,9% e outro considera que 90% já é suficiente.

Verificáveis, isto é, deve ser possível testar se os requisitos estão sendo atendidos.

Um contra-exemplo é um requisito que apenas requer que o sistema seja amigável. Como os desenvolvedores vão saber se estão atendendo a essa expectativa dos clientes?

Engenharia de Requisitos

» Priorizados;

Por fim, os requisitos devem ser priorizados.

Às vezes, o termo requisitos é interpretado de forma literal, isto é, como uma **lista de funcionalidades e restrições** obrigatórias em sistemas de software.

Nem sempre aquilo que é especificado pelos clientes será implementado nas releases iniciais.

Exemplo: restrições de prazo e custos podem postergar a implementação de certos requisitos.

Engenharia de Requisitos

Engenharia de Requisitos é uma atividade **multidisciplinar e complexa**.

Exemplos:

Fatores políticos podem fazer com que certos stakeholders não colaborem com a elicitação de requisitos que ameacem seu poder e status na organização.

Outros stakeholders simplesmente podem **não ter tempo** para se reunir com os desenvolvedores, a fim de explicar os requisitos do sistema.

A especificação de requisitos pode ser impactada ainda por uma **barreira cognitiva** entre os stakeholders e desenvolvedores.

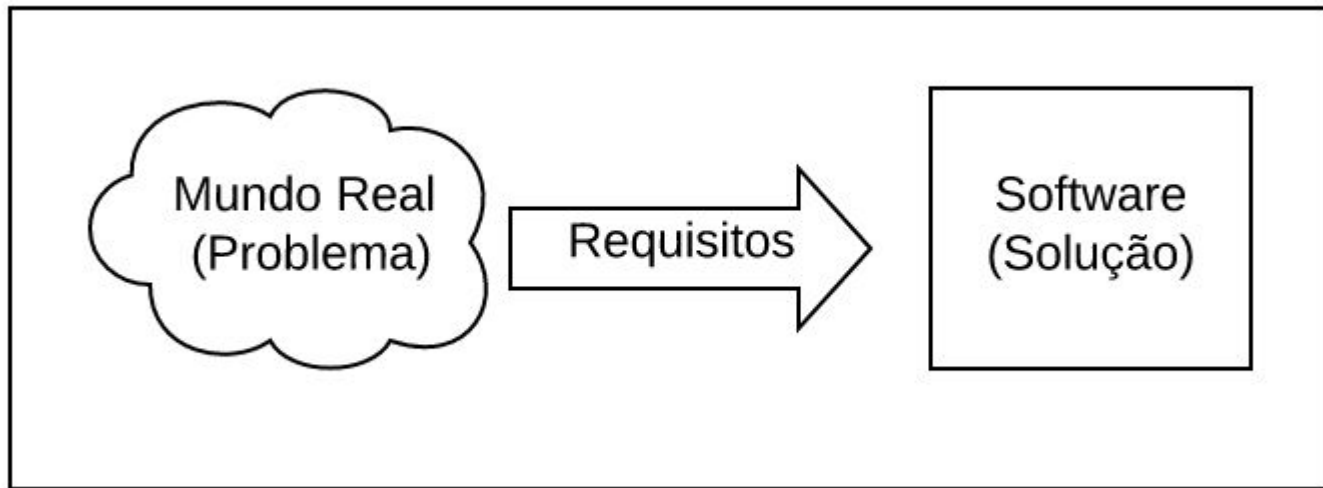
Principais Desafios

Em 2016, cerca de duas dezenas de pesquisadores coordenaram um survey com 228 empresas, distribuídas por 10 países, incluindo o Brasil. Quando questionadas sobre os principais problemas enfrentados na especificação de requisitos, as dez respostas mais comuns foram as seguintes (incluindo o percentual de empresas que apontou cada problema):

- » Requisitos incompletos ou não-documentados (48%)
- » Falhas de comunicação entre membros do time e os clientes (41%)
- » Requisitos em constante mudança (33%)
- » Requisitos especificados de forma abstrata (33%)
- » Restrições de tempo (32%)
- » Problemas de comunicação entre membros do time (27%)
- » Stakeholders com dificuldade de separar requisitos e soluções (25%)
- » Falta de apoio dos clientes (20%)

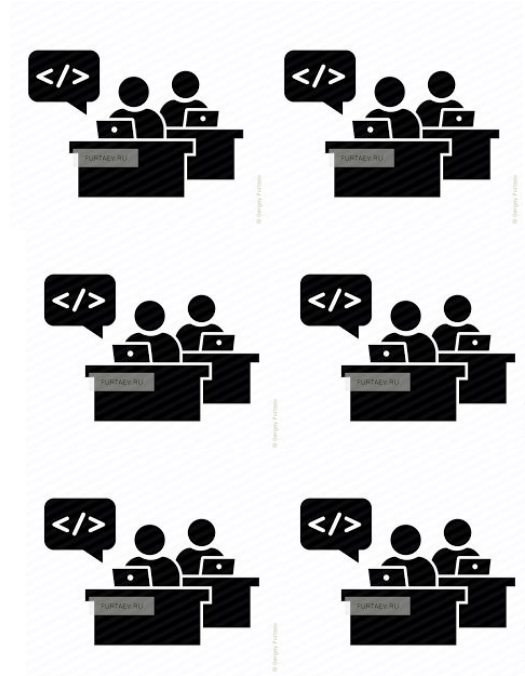
Requisitos - RESUMO

Os requisitos são a **ponte** que liga um problema do mundo real a um sistema de software que o soluciona.



Histórias de Usuários

Antes ...



Analistas

(PRD: program requirement document)

Programadores
(Fábrica de Software)

Hoje ...



Product Owner senta junto dos desenvolvedores e
"explica" requisitos para eles

Histórias de Usuários - 3C's

» Cartão + Conversas + Confirmação

Cartão: usado pelos clientes para escrever, na sua linguagem e em poucas sentenças, uma funcionalidade que esperam ver implementada no sistema.

Conversas entre clientes e desenvolvedores, por meio das quais os clientes explicam e detalham o que escreveram em cada cartão.

Histórias de Usuários - 3C's

» **C**artão + **C**onversas + **C**onfirmação

Confirmação: é basicamente um teste de alto nível — especificado pelo cliente — para verificar se a história foi implementada conforme esperado.

Descrição dos cenários, exemplos e casos de teste que o cliente irá usar para confirmar a implementação da história.

São também chamados de testes de aceitação de histórias.

Eles devem ser escritos o quanto antes, preferencialmente no início de uma iteração.

Histórias de Usuários - 3C's

Exemplo: Loja Virtual

- » **Cartão:** "Fechar uma compra"
- » **Conversas:** PO explica os meios de pagamento; as formas de entrega, formas de parcelamento, etc
- » **Confirmação:**
 - Testar compra à vista e compra parcelado
 - Testar com cartões de crédito A, B e C
 - Testar com modos de entrega X e Y

Histórias de Usuários - 3C's

Especificações de requisitos por meio de histórias **não consistem** apenas de **duas ou três sentenças**.

A história que se escreve é um lembrete do representante dos clientes para os desenvolvedores.

Durante todo o sprint ele se compromete a estar disponível para refinar a história e explicá-la para os desenvolvedores.

Também se compromete a considerar a história implementada desde que ela satisfaça os testes de confirmação que ele mesmo especificou.

Histórias de Usuários - Resumo

Quando usamos histórias de usuários, **atividades de Engenharia de Requisitos** ocorrem ao **longo de todo o desenvolvimento**.

Troca-se um **documento** de requisitos com centenas de páginas por **conversas frequentes**.

São também compatíveis com os **princípios do Manifesto Ágil**:

Indivíduos e interações, mais do que processos e ferramentas; software em funcionamento, mais do que documentação abrangente; colaboração com o cliente, mais do que negociação de contratos; resposta a mudanças, mais do que seguir um plano.

Boas características (INVEST)

- » **I**ndependentes
- » Abertas para **N**egociação
- » Agregar **V**alor
- » **E**stimáveis
- » **S**ucintas
- » **T**estáveis

Histórias de Usuários - Boas características

Histórias devem ser **independentes**:

- » Dadas duas histórias X e Y, deve ser possível implementá-las em qualquer ordem.

Histórias devem ser **abertas para negociação**:

- » São convites para **conversas entre clientes e desenvolvedores** durante um sprint.
- » Desenvolvedores devem estar abertos para implementar **detalhes** que **não estão expressos**.
- » E os clientes devem aceitar **argumentos técnicos** vindos dos desenvolvedores.

Histórias de Usuários - Boas características

Histórias devem agregar **valor** para o negócio dos clientes:

- » Histórias são propostas, escritas e priorizadas pelos clientes e de acordo com o valor que elas agregam ao seu negócio.
- » Não existe a figura de uma história técnica, como a seguinte: o sistema deve ser implementado usando React.

Deve ser viável **estimar** o tamanho de uma história:

- » Ex: quantos dias serão necessários para implementá-la.

Histórias de Usuários - Boas características

Histórias devem ser **sucintas**:

- » Para **facilitar** o **entendimento** e **estimativa** das mesmas.
- » Deve ser possível implementar as histórias do topo do backlog em menos de uma semana.
- » Até se **admite** histórias **complexas** e **grandes**, as quais são chamadas de **épicas**.
 - » Elas ficam posicionadas no **final do backlog**.
 - » Exemplo: um fórum de perguntas e respostas, como o Stack Overflow.
 - "implementar uma versão de X para celulares, Android e iOS".

Histórias de Usuários - Boas características

Histórias devem ser **testáveis**, isto é, elas devem ter critérios de aceitação objetivos:

- » Ex.: o cliente pode pagar com cartões de crédito.
 - » Uma vez definidas as bandeiras de cartões de crédito que serão aceitas, essa história é testável.
- » Contra-exemplo: um cliente não deve esperar muito para ter sua compra confirmada.
 - » Essa é uma história vaga e, portanto, com um critério de aceitação também vago.

Histórias de Usuários - Papéis

Antes de começar a escrever histórias, recomenda-se **listar os principais usuários** que vão interagir com o sistema.

Assim, evita-se que as histórias fiquem **enviesadas** e atendam às necessidades de apenas certos usuários.

Definidos esses papéis de usuários (user roles), costuma-se escrever as histórias no seguinte formato:

Como um [papel de usuário], eu gostaria de [realizar algo com o sistema]

Quem escreve as histórias?

- » PO (product owner)
- » **Workshop** de escrita de histórias
 - Realizado no início do projeto
 - Objetivo: criar uma lista inicial de histórias
 - Participam representantes dos principais usuários



Exemplo: Sistema de controle de bibliotecas

3 tipos de usuários

- » Usuário típico
- » Professor
- » Funcionário da biblioteca

Histórias do usuário típico

- » Como usuário típico, eu gostaria de realizar empréstimos de livros
- » Como usuário típico, eu gostaria de devolver um livro que tomei emprestado
- » Como usuário típico, eu gostaria de renovar empréstimos de livros

Histórias do usuário típico

- » Como usuário típico, eu gostaria de pesquisar por livros
- » Como usuário típico, eu gostaria de reservar livros que estão emprestados
- » Como usuário típico, eu gostaria de receber e-mails com novas aquisições

Histórias de professores

- » Como professor, eu gostaria de realizar empréstimos de maior duração
- » Como professor, eu gostaria de sugerir a compra de livros
- » Como professor, eu gostaria de doar livros para a biblioteca
- » Como professor, eu gostaria de devolver livros em outras bibliotecas

Histórias de funcionários da biblioteca

- » Como funcionário, eu gostaria de cadastrar novos usuários
- » Como funcionário, eu gostaria de cadastrar novos livros
- » Como funcionário, eu gostaria de dar baixa em livros estragados
- » Como funcionário, eu gostaria de obter estatísticas sobre o acervo

Histórias de funcionários da biblioteca

- » Como funcionário, eu gostaria que o sistema envie e-mails de cobrança para alunos com empréstimos atrasados
- » Como funcionário, eu gostaria que o sistema aplicasse multas quando da devolução de empréstimos atrasados

**Histórias ↔
Requisitos Funcionais**

**E os requisitos
não-funcionais?**



Requisitos não Funcionais (RNF)

- » Time deve definir RNF com o Product Owner
- » Time deve incluir RNF nos **critérios de conclusão de um sprint** (done criteria)

Exemplo: Suponha que **desempenho** seja um RNF importante

- » Pode-se definir que história para ser considerada pronta deve:
 - Passar por uma **revisão de código** focada em desempenho
 - Passar por **testes de desempenho**, com carga real